

Протокол Virtual Data Bus (VDB)

Версия документа: 1.0
Дата: 03 апреля 2025 г.

Введение

Протокол **Virtual Data Bus (VDB)** разработан для обмена данными и управления устройствами в рамках единой системы, позволяя им обмениваться данными и управлять, друг другом. Он обеспечивает взаимодействие между разнородными устройствами, такими как персональные компьютеры, одноплатные компьютеры и микроконтроллеры, через различные физические интерфейсы с поддержкой транзитной передачи сообщений. VDB является протоколом типа "**peer-to-peer**", что означает отсутствие центрального контроллера и равноправие всех устройств.

1. Основные характеристики

- Равноправие устройств: Все устройства в системе равноправны, центрального узла управления нет.
 - Адресация: Статическая, задается вручную. Максимальное количество устройств — 255. Адреса уникальны и формируются путем увеличения последнего использованного адреса на единицу.
 - Функционал: поддерживает операции чтения и записи данных, подтверждение доставки сообщений, обработку ошибок.
 - Проверка целостности: используется CRC на уровне физического интерфейса.
-

2. Структура сообщения

Каждое сообщение в протоколе VDB состоит из **заголовка** фиксированной длины 8 байт и необязательного **поля данных**. Структура сообщения представлена в таблице ниже:

Поле	Размер (байт)	Смещение (байт)	Описание
cmd	1	0	Тип команды (например, запрос на чтение или запись)
status	1	1	Статус запроса или ответа (успех/ошибка)
src			Адрес отправителя
function	1	2	Адрес устройства отправителя
device	1	3	Адрес функции отправителя
dst			Адрес получателя
function	1	4	Адрес устройства получателя
device	1	5	Адрес функции получателя
byteCount	2	6	Количество байтов в поле данных
payload	0 ... n	8	Поле данных, состоящее из байтов

Дополнительные пояснения:

- **Поля src и dst:** состоят из двух байт — первый указывает адрес устройства, второй — адрес функции внутри устройства.
 - **Поле byteCount:** указывает количество байтов в payload. Если равно 0, поле данных отсутствует.
 - **Поле payload:** содержит произвольное количество байт данных, для некоторых устройств может быть ограничен максимальный размер **payload**.
 - **Проверка целостности:** CRC добавляется на уровне физического интерфейса и не входит в структуру сообщения, описанную выше.
-

3. Типы команд и состояний

Команды (cmd)

Поле cmd определяет тип операции и принимает следующие значения:

Значение	Описание
0x00	Запрос на запись
0x80	Ответ на запись
0x01	Запрос на чтение
0x81	Ответ на чтение

Статусы (status)

Поле status указывает состояние запроса или ответа:

Значение	Описание
0x00	Запрос без ответа
0x80	Запрос с ответом или успешный ответ
0x80 + код ошибки	Ошибка (например, 0x81 — ошибка доставки)

Коды ошибок

При возникновении ошибки к базовому значению 0x80 добавляется код ошибки:

Значение	Описание
0x7F	Работает заглушка (для отладки)
0x01	Ошибка доставки сообщения
0x02	Превышен размер буфера
0x03	Устройство не существует
0x04	Функция не существует
Дополнительные коды ошибок могут добавляться в отдельном протоколе	

4. Логика функционирования протокола

Протокол VDB работает следующим образом:

1. Запрос на запись:

- Устройство формирует сообщение, указывая: cmd = 0x00, status, src, dst и, при необходимости, payload.
- Если требуется подтверждение выполнения, устанавливается status = 0x80. Для некритичных команд используется status = 0x00.
- Сообщение отправляется получателю напрямую или через промежуточные устройства по статическим таблицам маршрутизации.

2. Запрос на чтение:

- Устройство формирует сообщение, указывая: cmd = 0x01, status = 0x80, src, dst. Payload = 0 для запроса на чтение всегда устанавливается.
- Установка status = 0x00 приведёт к ошибке.
- Сообщение отправляется получателю напрямую или через промежуточные устройства по статическим таблицам маршрутизации.

3. Транзит сообщений:

- Транзитные устройства перенаправляют сообщение на основе статических таблиц маршрутов.
- Если доставка невозможна (например, устройство-получатель не существует), при запросе с ответом формируется сообщение с кодом ошибки.

4. Обработка сообщения:

- Получатель проверяет поле dst.device. Если адрес устройства совпадает, сообщение обрабатывается; иначе — пересылается дальше.
- Устройство проверяет наличие функции соответствующей адресу dst.function. Если функция существует, то функция выполняется.
- Если команда требует ответа, то при успешной обработке формируется ответ с status = 0x80. В случае ошибки указывается соответствующий код (например, 0x81 для ошибки доставки).

5. Формирование ответа:

- В ответном сообщении поля src и dst меняются местами.
- Поле cmd устанавливается в значение ответа (например, 0x80 для ответа на запись).
- Поле status отражает результат выполнения команды.

6. Обработка временной недоступности:

- Недоступность устройства проверяется через запросы с ответом. При отсутствии ответа отправитель может повторить запрос через определенный интервал.

5. Обработка ошибок и нештатных ситуаций

Протокол предусматривает следующие механизмы обработки ошибок:

- **Превышение размера payload:**
 - Если размер payload превышен:
 - Запрос с ответом: сообщение отбрасывается, формируется ошибка (status = 0x82).
 - Запрос без ответа: сообщение отбрасывается без уведомления.
 - **Устройство не найдено:**
 - При недоступности устройства-получателя возвращается status = 0x83 (для запросов с ответом) устройством, в котором была обнаружена невозможность дальнейшей передачи сообщения.
 - **Функция не найдена:**
 - Если функция отсутствует, возвращается status = 0x84.
 - **Временная недоступность:**
 - Отправитель может повторить запрос через заданный интервал.
 - **Сбой транзитного устройства:**
 - При сбое промежуточного устройства доставка может быть нарушена, устройства переходят в режим ожидания.
-

6. Планы на будущее (TO-DO)

Для улучшения протокола планируется:

- Внедрить **резервирование корневого устройства** или децентрализованный сбор телеметрии для повышения надежности.
- Разработать **мониторинг потерь запросов** в реальном времени.
- Определить метод **уведомления оператора** о прерываниях телеметрии.
- Ввести дополнительные проверки целостности данных (например, **ЕСС** для памяти).
- Провести анализ потребностей в модернизации системы и разработать план действий.